

AFRILANG Stdlib

std/c/graphcc

Bibliothèque AFRILANG `std/c/graphcc` (niveau complex, catégorie complex). Elle expose 7 fonction(s) :
connectedComponen

```
`import "std/c/graphcc" . `use graphcc`
```

- `connectedComponents(adj list number, n number) ? list number`
- `componentCount(adj list number, n number) ? number`
- `largestComponentSize(adj list number, n number) ? number`
- `sameComponent(adj list number, n number, a number, b number) ? bool`
- `componentSizes(adj list number, n number) ? list number`
- `outDegreeSum(adj list number, n number) ? list number`
- `isolateCount(adj list number, n number) ? number`

Category: complex | Tier: complex | Functions: 7

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/c/graphcc` (niveau complex, catégorie complex). Elle expose 7 fonction(s) : `connectedComponent`

```
`import "std/c/graphcc" . `use graphcc`  
- `connectedComponents(adj list number, n number) ? list number`  
- `componentCount(adj list number, n number) ? number`  
- `largestComponentSize(adj list number, n number) ? number`  
- `sameComponent(adj list number, n number, a number, b number) ? bool`  
- `componentSizes(adj list number, n number) ? list number`  
- `outDegreeSum(adj list number, n number) ? list number`  
- `isolateCount(adj list number, n number) ? number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/graphcc"  
use graphcc
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? connectedComponents(adj list number, n number) ? list  
? componentCount(adj list number, n number) ? number  
? largestComponentSize(adj list number, n number) ? number  
? sameComponent(adj list number, n number, a number, b number) ? bool  
? componentSizes(adj list number, n number) ? list  
? outDegreeSum(adj list number, n number) ? list  
? isolateCount(adj list number, n number) ? number
```

4. Exemples d'utilisation

```
import "std/c/graphcc"  
use graphcc  
  
create result = connectedComponents(/* args */)   
say result  
  
create result = componentCount(/* args */)   
say result  
  
create result = largestComponentSize(/* args */)   
say result  
  
create result = sameComponent(/* args */)   
say result  
  
create result = componentSizes(/* args */)   
say result  
  
create result = outDegreeSum(/* args */)   
say result
```

5. Bonnes pratiques

- ? Préférez `import "std/c/graphcc"` en tête de fichier.
- ? Lancez `afrilang check` avant de livrer.
- ? Combinez avec les modules stdlib de la même catégorie.
- ? Voir /stdlib/ pour le source live et les démos playground.
- ? Signalez les problèmes sur GitHub si une export manque.

6. Annexe ? source

module graphcc

```
function connectedComponents(adj list number, n number) returns list number
end

function componentCount(adj list number, n number) returns number
end

function largestComponentSize(adj list number, n number) returns number
end

function sameComponent(adj list number, n number, a number, b number) returns bool
end

function componentSizes(adj list number, n number) returns list number
end

function outDegreeSum(adj list number, n number) returns list number
end

function isolateCount(adj list number, n number) returns number
end
end
```

ENGLISH

1. Overview

AFRILANG library `std/c/graphcc` (tier complex, category complex). Exports 7 function(s):
connectedComponents, component

```
`import "std/c/graphcc" . `use graphcc`  
- `connectedComponents(adj list number, n number) ? list number`  
- `componentCount(adj list number, n number) ? number`  
- `largestComponentSize(adj list number, n number) ? number`  
- `sameComponent(adj list number, n number, a number, b number) ? bool`  
- `componentSizes(adj list number, n number) ? list number`  
- `outDegreeSum(adj list number, n number) ? list number`  
- `isolateCount(adj list number, n number) ? number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/graphcc"  
use graphcc
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? connectedComponents(adj list number, n number) ? list  
? componentCount(adj list number, n number) ? number  
? largestComponentSize(adj list number, n number) ? number  
? sameComponent(adj list number, n number, a number, b number) ? bool  
? componentSizes(adj list number, n number) ? list  
? outDegreeSum(adj list number, n number) ? list  
? isolateCount(adj list number, n number) ? number
```

4. Usage examples

```
import "std/c/graphcc"  
use graphcc  
  
create result = connectedComponents(/* args */)   
say result  
  
create result = componentCount(/* args */)   
say result  
  
create result = largestComponentSize(/* args */)   
say result  
  
create result = sameComponent(/* args */)   
say result  
  
create result = componentSizes(/* args */)   
say result  
  
create result = outDegreeSum(/* args */)   
say result
```

5. Best practices

```
? Prefer `import "std/c/graphcc"` at the top of your file.  
? Run `afriLang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module graphcc
```

```
function connectedComponents(adj list number, n number) returns list number
end

function componentCount(adj list number, n number) returns number
end

function largestComponentSize(adj list number, n number) returns number
end

function sameComponent(adj list number, n number, a number, b number) returns bool
end

function componentSizes(adj list number, n number) returns list number
end

function outDegreeSum(adj list number, n number) returns list number
end

function isolateCount(adj list number, n number) returns number
end
end
```